

*Let me explain **async/await**
in JavaScript in the easiest
way possible.*

*By default, JavaScript
program runs **line by line**,
top to bottom.*

But...

*JavaScript doesn't wait for
slow tasks.*

*If something takes time
like fetching data from a
server or waiting for a
timer...*

JavaScript says:

*"Okay, you take your time.
I'm moving to the next line."*

But it creates a problem.
Because the next line might
depend on the result of that
time-taking task.

And if JS already moved
ahead, you're left with
undefined or broken logic.

This is what we mean when we say the **normal flow breaks**.

Let me explain it to you with an example

```
console.log("1. Start");
```

```
setTimeout(() => {  
  console.log("2. Fetched data  
(simulated)");  
}, 2000);
```

```
console.log("3. End");
```


**You might think this will
print:**

- 1. Start**
- 2. Fetched data (simulated)**
- 3. End**

But actual output will be:

1. Start

3. End

2. Fetched Data(simulated)

Why?

Because setTimeout takes time, and JavaScript doesn't wait.

It logs Step 1, **skips over the timer**, logs Step 3, and later prints Step 2 when the 2 seconds are over.

JavaScript doesn't wait by default.

But with **async/await**, we tell it to hold on until something finishes (like fetching data).

Now can wait for Step 2 before moving to Step 3

Now here's how to make JavaScript wait

```
async function fetchData() {  
  console.log("1. Start");  
  
  await new Promise(resolve => {  
    setTimeout(() => {  
      console.log("2. Fetched data  
(simulated)");  
      resolve();  
    }, 2000);  
  });  
  
  console.log("3. End");  
}  
  
fetchData();
```

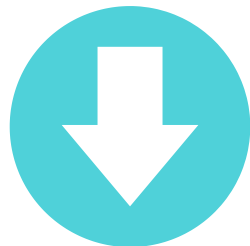
Now, the output will be:

- 1. Start**
- 2. Fetched data (simulated)**
- 3. End**

 In real-world apps, we use `async/await` when:

- Fetching data from APIs (e.g. weather, user info, product data)
- Talking to a database
- Reading from a file (in Node.js)
- Waiting for timers or animations

React



If it was Helpful